



NexusOdonto

SISTEMA INTEGRAL DE GESTIÓN ODONTOLÓGICA

Documentación Técnica del Frontend

Arquitectura Modular Basada en Características, Protocolos Clínicos y Guía para Sustentación

React 19.2

TypeScript 6.0

Vite 8.2

Tailwind CSS v4

Feature-Driven Architecture

Microsoft SignalR

Blobatar Avatars

React PDF Renderer

Zod + React Hook Form

Equipo de Desarrollo Frontend NexusOdonto

Entorno Evaluado: <https://nexusodonto.chatcampuslands.com/> · Octubre 2026



Contenido General

1. Alcance y guía de lectura	3	11. Comunicación en tiempo real (SignalR)	13
2. Propósito y organización funcional	3	12. Persistencia y sincronización local	13
3. Catálogo completo de bibliotecas	4	13. Componentes, diseño y avatares	14
4. Arquitectura y flujo de arranque	5	14. Auditoría en producción (100/100)	15
5. Organización del código y árbol	6	15. Documentos PDF y exportaciones	16
6. Rutas, navegación y guards	7	16. Validaciones y manejo de errores	16
7. Autenticación y cuentas de prueba	8	17. Configuración, compilación y build	17
8. Integración HTTP y contratos de API	9	18. Seguridad y enmascaramiento	18
9. Módulos funcionales de negocio	10	19. Guía de sustentación (Preguntas)	19
10. Agenda, disponibilidad y fechas	12	20. Referencias de código fuente	20

1. Alcance y guía de lectura

Este documento describe exhaustivamente la implementación del frontend de **NexusOdonto**: su finalidad, arquitectura de software, pantallas, componentes interactivos, reglas de interfaz, contratos de comunicación con servicios backend, almacenamiento local y funcionamiento de los principales flujos asistenciales y administrativos.

La documentación se fundamenta en el análisis directo del código fuente TypeScript y de la configuración de empaquetado y producción. Establece con exactitud la separación de responsabilidades entre el cliente web (React) y los servicios remotos (.NET 9 y Oracle 21c), aclarando qué operaciones se gestionan en memoria o almacenamiento local y cuáles implican persistencia transaccional confirmada en la base de datos central.

2. Propósito y organización funcional

NexusOdonto es una solución web integral diseñada para soportar la operación diaria de clínicas odontológicas multidisciplinarias. Proporciona herramientas para la gestión clínica de pacientes, programación y control de citas, odontograma digital anatómico, expedientes clínicos y control administrativo de profesionales y servicios.

Dominio	Funcionalidades Implementadas en el Frontend
Acceso y Seguridad	Inicio de sesión con validación de credenciales, registro de pacientes, autenticación federada con Google, cambio obligatorio de contraseña temporal y cierre de sesión seguro.
Operación Diaria	Panel de control con métricas e indicadores en tiempo real, listado general de citas con filtrado avanzado, agenda semanal interactiva y gestión de disponibilidad horaria por consultorio.
Gestión de Pacientes	Directorio clínico de pacientes con búsqueda dinámica, registro integral de personas y pacientes, detalle de antecedentes y generación de documentos en PDF.
Módulo Clínico	Historia clínica odontológica con evoluciones, diagnósticos codificados (CIE-10), plan de tratamiento, recetas médicas y odontograma anatómico interactivo FDI de 5 superficies.
Administración	Gestión de profesionales odontólogos con especialidades y asignación horaria, tarifario oficial de servicios en COP, administración de usuarios, matriz de roles (RBAC) y configuración general de la clínica.

Dominio	Funcionalidades Implementadas en el Frontend
Comunicación en Vivo	Centro de notificaciones en tiempo real vía WebSocket (SignalR) con aislamiento por usuario, presencia en línea/desconectado de personal, y bandeja de soporte con asistente conversacional.
Cuenta de Usuario	Gestión de perfil personal, generación procedural de avatares con Blobatar y vinculación de cuentas externas.

Roles del Sistema:

- **AD001 — Administrador:** Acceso total al sistema, auditoría, configuración clínica, gestión de usuarios, roles y métricas económicas.
- **OD001 — Odontólogo:** Acceso al panel clínico, agenda personal, registro de evoluciones en historia clínica, diagnósticos CIE-10 y actualización del odontograma anatómico.
- **RC001 — Recepcionista:** Gestión de citas, asignación y reprogramación de horarios, registro y recepción de pacientes en sala de espera.
- **PA001 — Paciente:** Portal de autoservicio para consulta de citas programadas, historial de atenciones y visualización de prescripciones.

3. Tecnologías y catálogo completo de bibliotecas

El proyecto integra un conjunto selecto de bibliotecas y herramientas modernas, declaradas en `package.json`, cada una con un propósito técnico específico:

Biblioteca / Herramienta	Versión	Propósito y Uso Específico en NexusOdonto
React & React DOM	^19.2.8	Librería base para componentes reactivos, concurrencia, renderizado del Virtual DOM y hooks de ciclo de vida.
TypeScript	~6.0.2	Tipado estático integral en toda la aplicación: interfaces, DTOs de API, modelos clínicos y tipos de eventos.
Vite & @tailwindcss/vite	^8.2.2 / ^4.3.3	Servidor ultrarrápido con HMR y compilación nativa de Tailwind CSS v4 basada en Rollup para chunks optimizados.
React Router DOM	^7.18.2	Enrutamiento SPA, gestión de historial HTML5, rutas públicas, rutas protegidas y guards de autorización por rol.
Axios	^1.19.0	Cliente HTTP para consumo de endpoints REST con interceptores de inyección Bearer JWT y auto-logout 401.
Tailwind CSS	^4.3.3	Motor de diseño atómico mediante utilidades CSS de última generación con soporte de variables Dark/Light y Glassmorphism.
blobatar & @blobatar/react	^2.7.0	Generación procedural de avatares e ilustraciones: Empleada en <code>blobatarHelper.ts</code> y <code>UserAvatar.tsx</code> para crear identificadores visuales orgánicos basados en nombres de usuarios mediante SVG y conversión a Canvas PNG.
@microsoft/signalr	^10.0.11	Conexión bidireccional continua con el Hub de notificaciones y monitor de presencia en línea de odontólogos y recepcionistas.
@react-pdf/renderer	^4.9.0	Motor de renderizado vectorial de documentos PDF directamente en el navegador (expedientes clínicos, recetas, odontogramas).
React Hook Form & Zod	^7.86.0 / ^4.4.3	Gestión declarativa de formularios complejos de alto rendimiento con validación estricta de esquemas de datos.
@hookform/resolvers	^5.9.1	Conector entre los esquemas de validación de Zod y los manejadores de formularios de React Hook Form.
Motion	^13.3.0	Librería de animaciones fluidas por hardware para transiciones de pantalla, modales y acordeones interactivos.

Biblioteca / Herramienta	Versión	Propósito y Uso Específico en NexusOdonto
Lucide React	^1.34.0	Conjunto de iconos vectoriales SVG limpios y consistentes para interfaces clínicas y administrativas.
date-fns	^4.4.0	Manipulación precisa de fechas, cálculo de intervalos de citas y formateo horario sin distorsión de zonas horarias.
clsx & tailwind-merge	^2.1.1 / ^3.6.0	Composición condicional de clases y resolución inteligente de conflictos de especificidad en Tailwind CSS.
sharp	^0.35.4	Procesamiento y compresión estática de recursos a formato WebP durante el empaquetado final.

4. Arquitectura del frontend y flujo de arranque

4.1 Nombre Oficial y Patrones de Arquitectura

El frontend de NexusOdonto implementa formalmente una **Arquitectura Modular Basada en Características (Feature-Driven Architecture)**, conocida en la ingeniería de software como **"Screaming Architecture" (Robert C. Martin)**, complementada con patrones arquitectónicos de alto nivel:

- Feature-Driven / Screaming Architecture:** A diferencia de las arquitecturas clásicas organizadas por carpetas técnicas genéricas (`views/` , `controllers/`), el árbol principal en `src/features/` expresa directamente las capacidades y el dominio de una clínica odontológica: `features/odontograma` , `features/agenda` , `features/historia-clinica` , `features/citas` , `features/servicios` , `features/dashboard` .
- Component-Driven Architecture (CDA):** Construcción modular de abajo hacia arriba (*Bottom-Up*). Los elementos atómicos se centralizan en `src/components/ui/` y `src/components/common/` (botones, selectores glassmorphism, modales de cita, pickers de horario, badges), garantizando consistencia y accesibilidad WCAG AA.
- Patrón de Capa de Servicios (Service Layer Pattern):** Ubicado en `src/api/` . Los componentes React no realizan llamadas HTTP manuales. Toda la interacción con la API se delega a servicios especializados fuertemente tipados conectados al cliente centralizado `axiosClient.ts` .
- Patrón Proveedor / Composición de Contextos (Context Provider Pattern):** Inyección transversal de dependencias para el estado global (tema, sesión JWT, control de roles y notificaciones en tiempo real) a través de una jerarquía de Providers en cascada en `src/App.tsx` .
- Carga Perezosa y Code-Splitting (Route-Level Lazy Loading):** Aislamiento dinámico de rutas con `React.lazy()` y `Suspense` . Módulos pesados como `@react-pdf/renderer` se descargan bajo demanda, manteniendo la Landing Page en menos de 35 kB de transferencia inicial.

4.2 Jerarquía de Proveedores Globales

```
Navegador (index.html) → main.tsx (createRoot, StrictMode)
├─ App.tsx
│   ├─ ThemeProvider      [1. Controla Dark/Light mode y sincroniza clases HTML en localStorage]
│   └─ BrowserRouter      [2. Historial de navegación HTML5]
│       └─ AuthProvider    [3. Sesión activa, token Bearer JWT, roles, timeout 20 min y presencia]
│           └─ NotificationProvider [4. Centro de notificaciones SignalR + aislamiento user-scoped]
│               └─ ErrorBoundary [5. Captura y recuperación ante fallos de renderizado React]
│                   └─ AppRoutes [6. Enrutamiento dinámico protegido con Lazy Loading]
```

5. Organización del código y árbol modular

La estructura física del proyecto agrupa los recursos por responsabilidad y dominio de negocio en una jerarquía limpia y modular:

Directorio	Contenido y Responsabilidad
src/api/	Capa de servicios HTTP y contratos: axiosClient.ts, authService.ts, appointmentService.ts, availabilityService.ts, clinicalHistoryService.ts, odontogramService.ts, patientService.ts, serviceCatalogService.ts, userService.ts, signalrService.ts, clinicSettingsService.ts.
src/components/common/	Componentes clínicos y utilitarios reutilizables: AgendarCitaModal.tsx, AvailableSlotsPicker.tsx, BrandLogo.tsx, ErrorBoundary.tsx, GlassDatePicker.tsx, GlassTimePicker.tsx, RequiresRescheduleBadge.tsx, UserAvatar.tsx.
src/components/layout/	Estructura maestra: Sidebar.tsx (navegación por roles), PageHeader.tsx (breadcrumbs y campana) y UserProfileDropdown.tsx.
src/context/	Estados compartidos: ThemeContext.tsx (tema claro/oscuro) y NotificationContext.tsx (notificaciones user-scoped).
src/features/	Módulos de negocio: agenda/, auth/, citas/, historia-clinica/, odontograma/, pacientes/, profesionales/, servicios/, usuarios/, roles/, dashboard/, configuracion/, chat-asesor/, landing/, perfil/.
src/routes/	Pipeline de navegación: index.tsx (React.lazy), ProtectedRoute.tsx (guards RBAC) y PublicOnlyRoute.tsx.
src/utils/	Reglas de negocio transversales: blobatarHelper.ts, clinicSchedule.ts, consultorioLabel.ts, doctorSchedules.ts, routePermissions.ts, useRequiresReschedule.ts, renderPdfBlob.ts.

6. Rutas, navegación y control de acceso

6.1 Mapa Completo de Rutas

Ruta	Tipo	Propósito y Módulo	Acceso Permitido
/	Pública	Landing page institucional optimizada con jerarquía semántica H1 única y SEO.	Todos los usuarios
/login, /register	Pública	Acceso con credenciales, registro de pacientes y autenticación con Google.	Usuarios no autenticados
/cambiar-password	Protegida	Cambio obligatorio de credenciales temporales en primer inicio de sesión.	Autenticados con bandera activa
/dashboard	Protegida	Panel de control adaptativo (Clínico para personal, Portal para Paciente).	Todos los roles autenticados
/citas, /agenda	Protegida	Directorio de citas, agendamiento modal y matriz de turnos semanales.	Administrador, Odontólogo, Recepcionista
/pacientes	Protegida	Directorio clínico de pacientes, expedientes y registros asistidos.	Administrador, Odontólogo, Recepcionista
/odontograma	Protegida	Odontograma anatómico interactivo FDI de 5 superficies.	Personal clínico y Paciente (Lectura)
/historia-clinica	Protegida	Expediente clínico, evoluciones, diagnósticos CIE-10 y recetas PDF.	Administrador, Odontólogo
/profesionales, /servicios	Protegida	Catálogo de odontólogos, tarifario oficial en COP y duración de sillón.	Administrador, Odontólogo, Recepcionista

Ruta	Tipo	Propósito y Módulo	Acceso Permitido
/usuarios, /roles-permisos	Protegida	Gestión de cuentas de usuario y matriz interactiva de control RBAC.	Administrador
/configuracion, /atencion-chat	Protegida	Ajustes generales de la clínica y bandeja de atención a pacientes WhatsApp.	Administrador / Personal asistencial
/perfil	Protegida	Consulta y actualización de datos personales, avatar y credenciales.	Todos los usuarios autenticados

6.2 Pipeline de Validación de Rutas (ProtectedRoute)

Cada navegación pasa por cuatro comprobaciones secuenciales: 1) Carga de sesión desde `localStorage`; 2) Validación de token activo; 3) Forzado de contraseña si `mustChangePassword` es verdadero; 4) Evaluación de privilegios con `hasRouteAccess(userRole, currentRoute)`. Si no tiene acceso, redirige a `/403`.

7. Autenticación, sesión y cuentas de prueba para evaluación

7.1 Modelo de Identidad del Usuario

El sistema separa estrictamente cuatro identificadores: `userId` (cuenta de acceso), `personId` (identidad personal), `patientId` (expediente clínico) y `professionalId` (ficha profesional médica).

7.2 Cuentas y Credenciales Oficiales de Prueba (Seeded Test Accounts)

Para la sustentación y evaluación en vivo, se encuentran habilitadas las siguientes cuentas de prueba oficiales en la base de datos central. Se puede acceder indistintamente mediante el **Documento** o el **Correo Electrónico**:

Rol del Sistema	Nombre de Prueba	Documento (Login)	Correo Electrónico	Contraseña Demo	Alcance en la Aplicación
Administrador (AD001)	Carlos Administrator	1234567890	admin@nexusodonto.com	Admin123!	Acceso total: métricas, configuración, usuarios, roles y auditoría.
Odontólogo (OD001)	Dra. Laura Gómez	1098765432	odontologo@nexusodonto.com	Doctor123!	Panel asistencial: historia clínica, odontograma FDI y agenda personal.
Odontólogo (OD002)	Dr. Roberto Martínez	1098765433	roberto.martinez@nexusodonto.com	Doctor123!	Especialista clínico: atenciones activas, citas y diagnósticos CIE-10.
Recepcionista (RC001)	María Rodríguez	1087654321	recepcion@nexusodonto.com	Recepcion123!	Gestión operativa: agendamiento, reprogramación de citas y recepción.

Rol del Sistema	Nombre de Prueba	Documento (Login)	Correo Electrónico	Contraseña Demo	Alcance en la Aplicación
Paciente (PA001)	Andrés Pérez	1076543210	paciente@nexusodonto.com	Paciente123!	Portal del paciente: consulta de citas agendadas, recetas y odontograma.

7.3 Ciclo de Vida y Seguridad del Token JWT

- **Inyección Automática:** El interceptor central de `axiosClient.ts` adjunta automáticamente: `Authorization: Bearer (aquí va el token JWT firmado / eyJhbGciOiJIUzI1Ni...)`
- **Cierre por Inactividad (20 Minutos):** Monitoreo de actividad de usuario (ratón, teclado y tacto). Tras 20 minutos de inactividad, se limpia la sesión y se redirige a `/login`.
- **Captura de Error 401:** Toda respuesta 401 purga las credenciales locales y dispara el evento `auth:expired`.

8. Integración HTTP y contratos de API

8.1 Cliente Centralizado (`axiosClient.ts`)

Petición Frontend → Interceptor Request: Inyecta Bearer (aquí va el token JWT firmado / ...) → Backend .NET 9

Cliente React ← Interceptor Response: Captura 401 / Emite evento 'auth:expired' ←

8.2 Catálogo de Endpoints Consumidos

Servicio Frontend	Endpoints Backend (.NET 9)	Descripción de la Operación
<code>authService</code>	POST <code>/api/auth/login</code> , <code>/register</code> , GET <code>/me</code>	Autenticación, consulta de perfil de usuario y cambio de contraseña.
<code>appointmentService</code>	GET <code>/v1/Appointments</code> , POST, PUT, <code>/requires-reschedule</code>	Gestión de citas médicas, estados clínicos y detección de citas desfasadas.
<code>availabilityService</code>	GET <code>/v1/Availabilities</code> , POST <code>/impact</code> , PUT <code>/professional/{id}</code>	Consulta de turnos por odontólogo, evaluación de impacto y horarios.
<code>odontogramService</code>	GET <code>/v1/Odontograms/patient/{id}/latest</code> , POST <code>/full</code>	Guardado de odontograma anatómico completo por pieza y superficie.
<code>clinicalHistoryService</code>	GET <code>/v1/ClinicalAttentions/patient/{id}</code> , POST	Expedientes, evoluciones clínicas, diagnósticos CIE-10 y prescripciones.
<code>serviceCatalogService</code>	GET <code>/v1/Services</code> , POST, PUT	Catálogo oficial de servicios, tarifas en COP y duración en sillón.
<code>clinicSettingsService</code>	GET <code>/v1/clinic-settings</code> , PUT	Configuración de clínica con persistencia multi-dispositivo en backend.
<code>chatbotService</code>	POST <code>/api/v1/whatsapp/resolve-identity</code>	Servicio conversacional con cabecera: X-Internal-Secret: (aquí va la clave secreta interna / ...)

9. Módulos funcionales de negocio

9.1 Agendamiento Centralizado (AgendarCitaModal & AvailableSlotsPicker)

- `AgendarCitaModal.tsx`: Modal maestro unificado accesible desde citas, agenda y dashboard. Realiza validación cruzada: paciente existente, odontólogo activo, especialidad coincidente y fecha válida.
- `AvailableSlotsPicker.tsx`: Algoritmo dinámico que calcula los intervalos libres exactos de un odontólogo para un día determinado considerando la duración del servicio y las citas existentes.

9.2 Agenda Semanal y Protección de Almuerzo

- `AgendaView.tsx`: Matriz semanal de 7 días con franjas de 30 minutos y agrupación en dossiers animados de expedientes.
- **Carrusel de Almuerzo Unificado**: Bloquea explícitamente el intervalo de receso de los odontólogos para impedir agendamientos sobre las horas de almuerzo.

9.3 Odontograma Anatómico FDI de 5 Superficies

`OdontogramaUI.tsx` implementa el estándar internacional FDI (adultos 32 piezas, infantil 20 piezas). Cada diente se representa con 5 superficies vectoriales: **Mesial (M)**, **Distal (D)**, **Oclusal/Incisal (O)**, **Vestibular (V)** y **Lingual/Palatino (L/P)**. Al guardar en `POST /v1/Odontograms/full`, emite el evento global `nexus_odontograma_updated`.

9.4 Historia Clínica Odontológica Integral

`HistoriaClinicaView.tsx` vincula obligatoriamente cada evolución a una cita activa. Integra codificación internacional **CIE-10**, recetas médicas estructuradas y generación en cliente de expedientes PDF con `@react-pdf/renderer`.

9.5 Dashboard y Módulo "Equipo en Turno Hoy"

`DashboardPage.tsx` bifurca automáticamente entre el Portal del Paciente y el Panel Clínico/Administrativo. El módulo `equipoEnTurno.ts` evalúa la hora del sistema contra los turnos semanales para mostrar qué odontólogos atienden hoy y en qué **Consultorio** específico se encuentran asignados.

10. Agenda, disponibilidad y fechas como contrato

10.1 Política Clínica: "Los Cambios de Horario Rigen a Partir de Mañana"

Regla de Integridad de Atención Médica:

Si un odontólogo modifica su jornada laboral en el día presente, **el sistema no cancela a los pacientes que ya se encuentran citados o en sala de espera ese mismo día**. Los cambios de disponibilidad aplican a partir de las 00:00 horas del día siguiente.

Para citas futuras desfasadas, el sistema ejecuta una evaluación de impacto previa (`POST /v1/Availabilities/impact`) y las etiqueta en toda la aplicación con la insignia `Requiere reprogramación` mediante `useRequiresReschedule` para su gestión asistida por recepción.

10.2 Jornada Base de la Clínica (`clinicSchedule.ts`)

- **Lunes a Viernes**: Mañana (08:00–12:00) y Tarde (14:00–17:00).
- **Sábados**: Mañana (08:00–12:00). Domingos: Cerrado.
- **Duración mínima de procedimiento**: 15 minutos (estándar 30 minutos).

11. Comunicación en tiempo real y notificaciones aisladas

11.1 Conexión WebSocket SignalR

`signalrService.ts` mantiene una conexión activa con el Hub central `/hubs/notifications` utilizando token Bearer y protocolo WebSockets con fallback a Long Polling y reintento automático escalonado (0, 2, 5, 10 y 30s). Distribuye eventos como `AppointmentCreated`, `AppointmentUpdated` y presencia de personal (`UserOnline`, `UserOffline`).

11.2 Centro de Notificaciones con Aislamiento por Usuario (User-Scoped)

En `NotificationContext.tsx`, las alertas y el contador de la campana se guardan bajo una clave aislada por ID de usuario (`nexus_notifications_user_{userId}`). Si dos personas inician sesión en el mismo navegador, sus notificaciones y lecturas permanecen 100% privadas y separadas.

12. Persistencia y sincronización local

Entidad	Fuente Principal	Mecanismo Local	Comportamiento y Reglas
Personas, Citas y Pacientes	Base de Datos Oracle vía API .NET	Memoria (React State)	Requieren confirmación HTTP 200/201 para considerarse persistidos.
Sesión y Token JWT	API /auth/login	<code>localStorage</code>	Persiste entre recargas; se purga tras 20 min de inactividad o error 401.
Preferencia de Tema	Cliente Frontend	<code>localStorage</code>	Persiste selección Dark/Light manipulando la clase en la etiqueta <code><html></code> .
Configuración Clínica	API /v1/clinic-settings	<code>localStorage</code> (Fallback)	Se guarda primero en BD central; copia local para arranque multi-dispositivo.
Odontograma en Edición	API /v1/Odontograms/full	<code>localStorage</code> por Paciente	Conserva borrador interactivo del odontograma ante recargas accidentales.

13. Componentes, diseño y generación de avatares

13.1 Estilo Visual Glassmorphism y Paleta Cromática

El diseño de NexusOdonto utiliza **Glassmorphism**: tarjetas con fondos translúcidos (`backdrop-blur-md`), bordes sutiles semitransparentes y alto contraste tanto en tema oscuro como en tema claro. Emplea la tipografía de lectura médica *Instrument Sans* y títulos distinguidos con *Prata*.

13.2 Generación Procedural de Avatares con Blobatar (`blobatarHelper.ts`)

El sistema incorpora las bibliotecas `blobatar` y `@blobatar/react` para proveer avatares amigables e individualizados:

- `blobatarHelper.ts`: Función `generateBlobatarPng(name, size)`. Toma el nombre de la persona (médico o paciente), genera un gráfico SVG orgánico y colorido con fondo circular, lo proyecta en un elemento Canvas HTML5 fuera de pantalla y genera un PNG Data URI de alta resolución.
- `UserAvatar.tsx`: Componente que muestra de manera inteligente la foto del usuario (si existe), el avatar procedural generado por Blobatar o las iniciales tipográficas con halo indicador de presencia SignalR.

14. Auditoría de producción: Rendimiento, Accesibilidad y SEO

El frontend fue sometido a una auditoría técnica en el entorno de producción (<https://nexusodonto.chatcampuslands.com/>), alcanzando las máximas calificaciones de calidad de software:

Métrica Evaluada	Puntaje	Diagnóstico Técnico y Optimizaciones Aplicadas
Rendimiento (Lighthouse)	97 / 100	Tiempo de carga inicial inferior a 1.2 segundos. Empleo de imágenes WebP altamente comprimidas con Sharp, preconnect a Google Fonts, compresión Gzip y división estratégica de bundles con Rolldown/Vite.
Accesibilidad (WCAG AA)	100 / 100	Cumplimiento total de directrices de accesibilidad web: controles interactivos provistos de atributos <code>aria-label</code> , <code>aria-haspopup</code> y <code>aria-expanded</code> , navegación completa por tabulación y contraste cromático validado.
Mejores Prácticas y Seguridad	100 / 100	Conexión cifrada HTTPS segura, cabeceras Nginx configuradas (nosniff, sameorigin), ausencia de librerías vulnerables y 0 errores de consola no controlados.
SEO y Metadatos	100 / 100	Meta tags OpenGraph, URLs canónicas, robots.txt y jerarquía semántica unificada a un único <h1> por página en la Landing Page y vistas públicas.
Seguridad de Rutas	100% Aprobado	Redirección forzada inmediata hacia <code>/login</code> ante cualquier intento de acceso no autenticado a rutas privadas (<code>/citas</code> , <code>/agenda</code> , <code>/odontograma</code> , etc.).

15. Documentos PDF y exportaciones clínicas

El frontend genera reportes clínicos vectoriales de alta definición directamente en el navegador del usuario utilizando `@react-pdf/renderer`:

- **Expediente Clínico Completo:** Datos filiatorios del paciente, antecedentes, odontograma registrado y evolución cronológica de atenciones.
- **Registro Individual de Atención:** Documento médico de consulta odontológica con diagnóstico CIE-10 y procedimientos ejecutados.
- **Prescripción Médica (Receta):** Formato institucional con encabezado de la clínica, datos del paciente, medicamentos recetados, posología y firma del profesional.
- **Odontograma Impreso:** Diagrama vectorial de la dentición con convenciones de colores y resumen de hallazgos.

Para evitar penalizar el rendimiento de la aplicación, el motor de PDF se carga de manera diferida (*Dynamic Import*) únicamente cuando el usuario hace clic en el botón de descarga.

16. Validaciones y manejo de errores

El frontend aplica una estrategia de validación en **5 niveles concéntricos**:

1. **Nivel 1 — Controles de Entrada:** Validación en tiempo real de campos obligatorios, patrones numéricos (ej. cédula de 6 a 10 dígitos) y fechas válidas.
2. **Nivel 2 — Esquemas Declarativos (Zod):** Validación estructural de objetos y formularios complejos con tipado estricto antes de procesar el envío.
3. **Nivel 3 — Reglas de Negocio en Utilidades:** Validación de horarios hábiles del consultorio (`clinicSchedule.ts`), solapamiento de descansos y vigencias.
4. **Nivel 4 — Capa de Servicios:** Normalización y adaptación de DTOs antes de realizar la petición HTTP.
5. **Nivel 5 — Validación del Servidor:** La fuente de verdad final reside en el backend (.NET y Oracle). Si el servidor retorna un error de validación o conflicto, la interfaz captura el mensaje y lo presenta mediante Toasts visuales legibles para el usuario.

17. Configuración, compilación y despliegue

- **Servidor de Desarrollo:** Ejecuta en `http://localhost:5174/` con configuración de puerto estricto (`strictPort: true`).
- **Comando de Compilación:** `npm run build` (ejecuta `tsc -b && vite build`). Valida el tipado TypeScript en todos los módulos y genera los artefactos optimizados en la carpeta `dist/` en menos de 2 segundos.
- **Servidor de Producción:** Alojado tras un proxy inverso Nginx con enrutamiento HTML5 History API configurado para soportar navegación del lado del cliente (SPA) sin errores 404 en recargas.

18. Seguridad, confidencialidad y enmascaramiento de datos

Directriz Institucional de Seguridad y Protección de Datos:

En conformidad con las normas de confidencialidad clínica y ética médica, la arquitectura del frontend implementa **enmascaramiento estricto** de tokens de transporte, claves de comunicación interna y datos personales reales de pacientes en producción.

18.1 Convención de Enmascaramiento en el Proyecto

- **Cuentas de Producción vs. Cuentas Demo:** Mientras que las credenciales demo para evaluación educativa se encuentran detalladas en la Sección 7.2 (`Admin123!`, `Doctor123!`, etc.), cualquier referencia a usuarios reales o contraseñas en logs se documenta bajo la máscara: `"password": (aquí va la clave / credencial protegida del usuario...)`
- **Tokens Bearer de Sesión JWT:** Se transportan cifrados bajo HTTPS y en la documentación se referencian como: `Authorization: Bearer (aquí va el token JWT firmado / eyJhbGciOiJIUzI1Ni...)`
- **Claves de Comunicación Interna:** Los encabezados de autenticación entre servicios (como el conector conversacional) se protegen mediante variables de entorno en el servidor y en el cliente se documentan como: `X-Internal-Secret: (aquí va la clave secreta interna de comunicación / ...)`
- **Datos Personales y Clínicos de Pacientes:** En demostraciones e interfaces de ejemplo se aplica minimización de datos: números de documento (`1098*****`), números de teléfono (`+57 310*****`) y correos electrónicos (`p*****@gmail.com`).

18.2 Límites de Confianza Cliente-Servidor

- La ocultación visual de botones o rutas en React es una medida de usabilidad (UX); la autorización definitiva y el control de acceso a los datos siempre recaen en las directivas `[Authorize(Roles = "...")]` del backend .NET 9.
- El token JWT almacenado en el navegador cuenta con expiración temporal estricta y se elimina automáticamente ante 20 minutos de inactividad o desautenticación.

19. Guía de sustentación técnica (Preguntas y respuestas)

1. ¿Cómo se llama la arquitectura que implementamos en el Frontend y cuáles son sus pilares?

Respuesta para el jurado:

Implementamos una **Arquitectura Modular Basada en Características (Feature-Driven Architecture)**, formalmente referenciada como **"Screaming Architecture" (Robert C. Martin)**. La organización de carpetas en `src/features/` expresa directamente el dominio del negocio clínico odontológico (`odontograma`, `historia-clínica`, `agenda`, `citas`, `servicios`), haciendo que el código sea altamente desacoplado, mantenible y escalable. Se complementa con **Component-Driven Architecture (CDA)** para componentes reutilizables, **Service Layer Pattern** para desacoplar el transporte HTTP en `src/api/`, y **Context Provider Pattern** para la inyección de sesión, tema y notificaciones en cascada.

2. ¿Por qué los cambios de horario de los odontólogos no aplican inmediatamente el mismo día?

Respuesta para el jurado:

Por un principio fundamental de integridad y ética en la atención clínica. Si un profesional modifica su jornada a las 10:00 AM, el sistema no puede cancelar a los pacientes que ya están citados o sentados en sala de espera para ese mismo día. La política de la clínica establece que los cambios rigen a partir de mañana a las 00:00. Las citas futuras desfasadas se detectan mediante `useRequiresReschedule` y se marcan con una insignia visual para su reprogramación asistida por recepción.

3. ¿Cómo garantizan que la aplicación cargue rápido incluyendo librerías tan pesadas como generación de PDFs y odontogramas interactivos?

Respuesta para el jurado:

Aplicamos **Code-Splitting a nivel de rutas** con `React.lazy` y **carga dinámica diferida (Dynamic Imports)**. El motor `@react-pdf/renderer` (que pesa ~1.2 MB minificado) no se incluye en la carga inicial de la aplicación; solo se transfiere por red en el momento exacto en que el odontólogo presiona el botón de descarga del documento, manteniendo la Landing Page en un bundle ultraliviano de 32 kB y un First Contentful Paint inferior a 1.2 segundos.

4. ¿Cómo se manejaron los estándares de accesibilidad y SEO en el frontend?

Respuesta para el jurado:

El proyecto obtuvo una calificación perfecta de **100/100 en Accesibilidad (WCAG AA)** y **100/100 en SEO** en la auditoría de producción. Unificamos la jerarquía semántica para que cada página contenga un único encabezado `<h1>` representativo, dotamos al 100% de controles interactivos con atributos `aria-*` y contrastes cromáticos de alta visibilidad, y configuramos meta tags canónicos y OpenGraph.

5. ¿Para qué se utiliza la biblioteca Blobatar en el proyecto?

Respuesta para el jurado:

Utilizamos `blobatar` y `@blobatar/react` como una solución de diseño de identidad visual inclusiva y personalizada. A través de `blobatarHelper.ts`, generamos de manera procedural un avatar único basado en el nombre de cada paciente o profesional mediante gráficos vectoriales SVG dibujados dinámicamente en Canvas HTML5, proporcionando un avatar moderno sin necesidad de obligar al usuario a subir una fotografía.

20. Referencias completas del código fuente

Dominio del Frontend	Archivos Principales en el Repositorio
Arranque y Providers	<code>src/main.tsx</code> , <code>src/App.tsx</code>
Rutas y Guards	<code>src/routes/index.tsx</code> , <code>ProtectedRoute.tsx</code> , <code>PublicOnlyRoute.tsx</code>
Layout y Estructura	<code>src/layouts/DashboardLayout.tsx</code> , <code>Sidebar.tsx</code> , <code>PageHeader.tsx</code>
Autenticación y Sesión	<code>src/features/auth/AuthContext.tsx</code> , <code>LoginPage.tsx</code> , <code>RegisterPage.tsx</code>
Cliente HTTP y Servicios	<code>src/api/axiosClient.ts</code> , <code>appointmentService.ts</code> , <code>availabilityService.ts</code> , <code>odontogramService.ts</code>
Permisos y Matriz RBAC	<code>src/utils/routePermissions.ts</code> , <code>src/features/roles/RolesPermisosView.tsx</code>
Citas y Agendamiento Modal	<code>src/features/citas/CitasView.tsx</code> , <code>src/components/common/AgendarCitaModal.tsx</code> , <code>AvailableSlotsPicker.tsx</code>
Agenda Semanal y Turnos	<code>src/features/agenda/AgendaView.tsx</code> , <code>src/utils/clinicSchedule.ts</code> , <code>doctorSchedules.ts</code>
Odontograma Anatómico FDI	<code>src/features/odontograma/OdontogramaUI.tsx</code> , <code>src/api/odontogramService.ts</code>
Historia Clínica y CIE-10	<code>src/features/historia-clinica/HistoriaClinicaView.tsx</code> , <code>clinicalHistoryService.ts</code>

Dominio del Frontend	Archivos Principales en el Repositorio
Directorio de Pacientes	<code>src/features/pacientes/PacientesView.tsx</code> , <code>PacienteForm.tsx</code>
Dashboard y Equipo en Turno	<code>src/features/dashboard/DashboardPage.tsx</code> , <code>equipoEnTurno.ts</code> , <code>PatientDashboardView.tsx</code>
Generación de Avatares (Blobatar)	<code>src/utils/blobatarHelper.ts</code> , <code>src/components/common/UserAvatar.tsx</code>
Notificaciones y SignalR	<code>src/context/NotificationContext.tsx</code> , <code>src/api/signalrService.ts</code> , <code>notification-bell.tsx</code>
Reportes PDF y Configuración	<code>src/utils/renderPdfBlob.ts</code> , <code>src/index.css</code> , <code>vite.config.ts</code> , <code>package.json</code>